

```
import cv2
import numpy as np
```

Convert a grayscale image to binary image

```
img = cv2.imread('./gray.jpeg', cv2.IMREAD_GRAYSCALE)
```

Using mean threshold value

```
mean = np.mean(img)
_, binary_img = cv2.threshold(img, mean, 255, cv2.THRESH_BINARY)
cv2.imwrite('./binary_image.jpeg', binary_img)
```

Output

True

=== Using user-defined threshold value

```
mean = int(input("Enter threshold value: "))
```

Output

Enter threshold value: 190

```
_, binary_img = cv2.threshold(img, mean, 255, cv2.THRESH_BINARY)
cv2.imwrite('./binary_image.jpeg', binary_img)
```

Output

True

= Convert a colored image to grayscale image

```
img = cv2.imread('./color.jpg')
```

By taking mean average of all three planes

```
grayscale = np.mean(img, axis=2)
# convert grayscale to unsigned integers (get rid of decimals/floating point numbers)
grayscale.astype(np.uint8)
```

Output

```
array([[214, 217, 221, ..., 162, 162, 162],
       [215, 217, 219, ..., 162, 162, 161],
       [217, 217, 219, ..., 162, 162, 161],
       ...,
       [184, 184, 184, ..., 197, 197, 197],
       [184, 184, 184, ..., 196, 195, 196],
       [184, 184, 184, ..., 195, 195, 195]],
      shape=(3004, 4103), dtype=uint8)
cv2.imwrite('./grayscale.jpg', grayscale)
```

Output

True

=== Input weightage from user Weightage is a value between 0 to 1 and sum of all three weightages is 1

```
weightage_r = 0.6
weightage_g = 0.0
weightage_b = 0.4
img = cv2.imread('./holi.jpg')
```

```
gray_img = (weightage_r * img[:, :, 2]) + (weightage_g * img[:, :, 1]) + (weightage_b * img[:, :, 0])
cv2.imwrite('./holi_grayscale.jpg', gray_img)
```

Output

True

= Draw a border around the image

```
img = cv2.imread('./gray.jpeg', cv2.IMREAD_GRAYSCALE)
# # check if image is binary
# flattened_values = (set(img.flatten()))
# if len(flattened_values) != 2:
#     # not a binary image
#     img = cv2.threshold(img, np.mean(img), 255, cv2.THRESH_BINARY)
print(img)
# img = np.array(img)

border_width = 200

img_with_border = cv2.copyMakeBorder(img, border_width, border_width, border_width, border_width, cv2.BORDER_CONSTANT, value=(255, 0, 0))
cv2.imwrite("border_image.jpeg", img_with_border)
```

Getting complement of image

```
img = cv2.imread('./binary_image.jpeg', cv2.IMREAD_GRAYSCALE)
complement = 255 - img
cv2.imwrite('complement.jpeg', complement)
```

Output

True